UNIVERSIDAD DE GUADALAJARA

CENTRO UNIVERSITARIO DE LOS VALLES

Licenciatura en Tecnologías de la Información



U2 - A4 APIs RESTfull con Flask

Materia: Desarrollo de Bases de Datos para Web.

NRC: 129430.

7° Semestre – Ciclo Escolar 2026B

Asesor: Adolfo Castillo Chavarín.

Alumno:
Ibarra Rodríguez Juan Pablo.

Fecha:
25 de octubre de 2026.

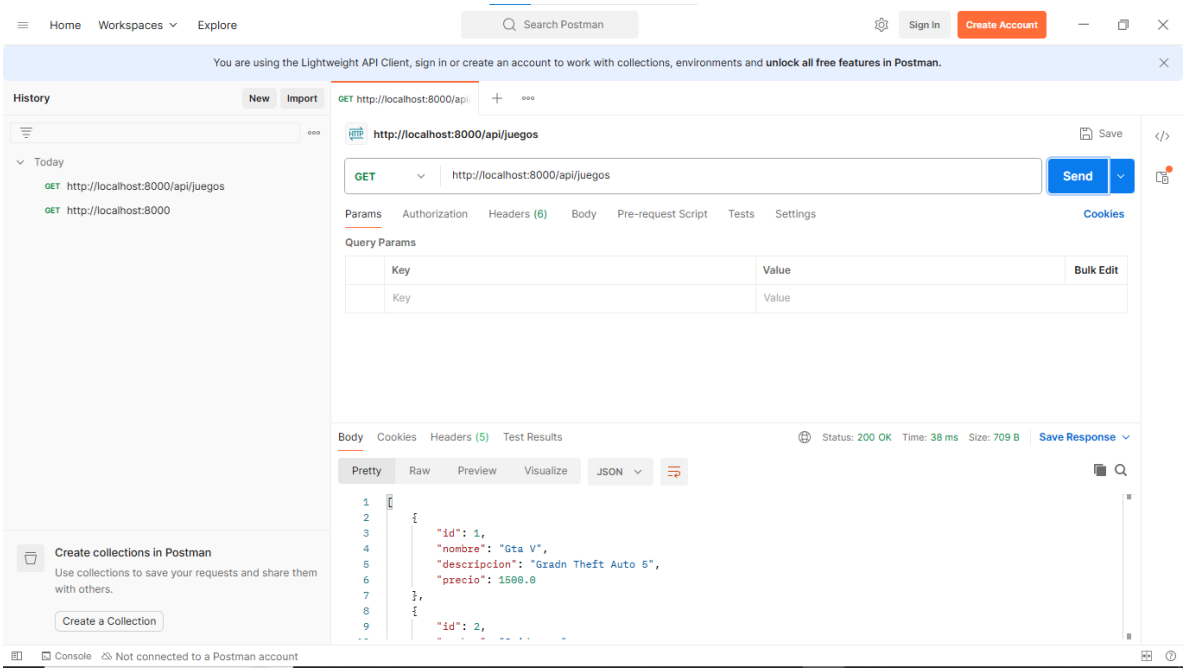
Reporte: U2 - A4 APIs RESTful con Flask

En esta actividad, integré una API RESTfull a la microaplicación de videojuegos que veníamos trabajando. El objetivo principal fue crear rutas específicas para que otras aplicaciones puedan consultar, agregar, modificar o eliminar juegos directamente usando formato JSON, sin necesidad de interactuar con las interfaces visuales (HTML).

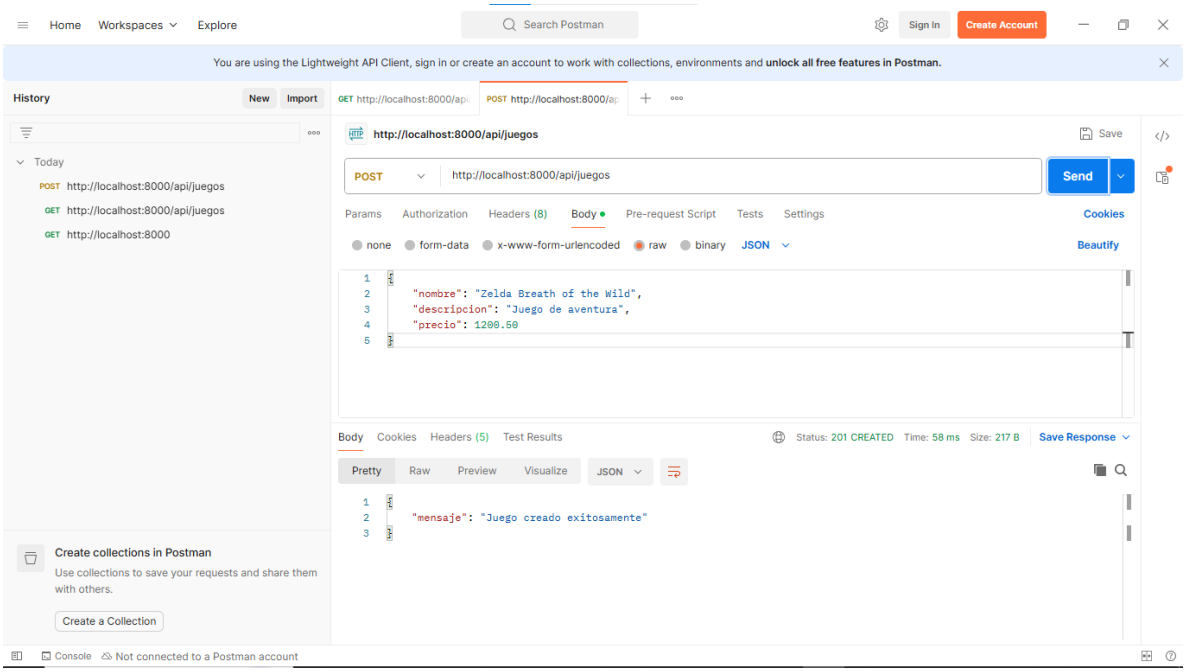
Descripción de los endpoints implementados y Pruebas.

Para lograr esto, utilicé la librería Flask-RESTfull y definí los siguientes endpoints:

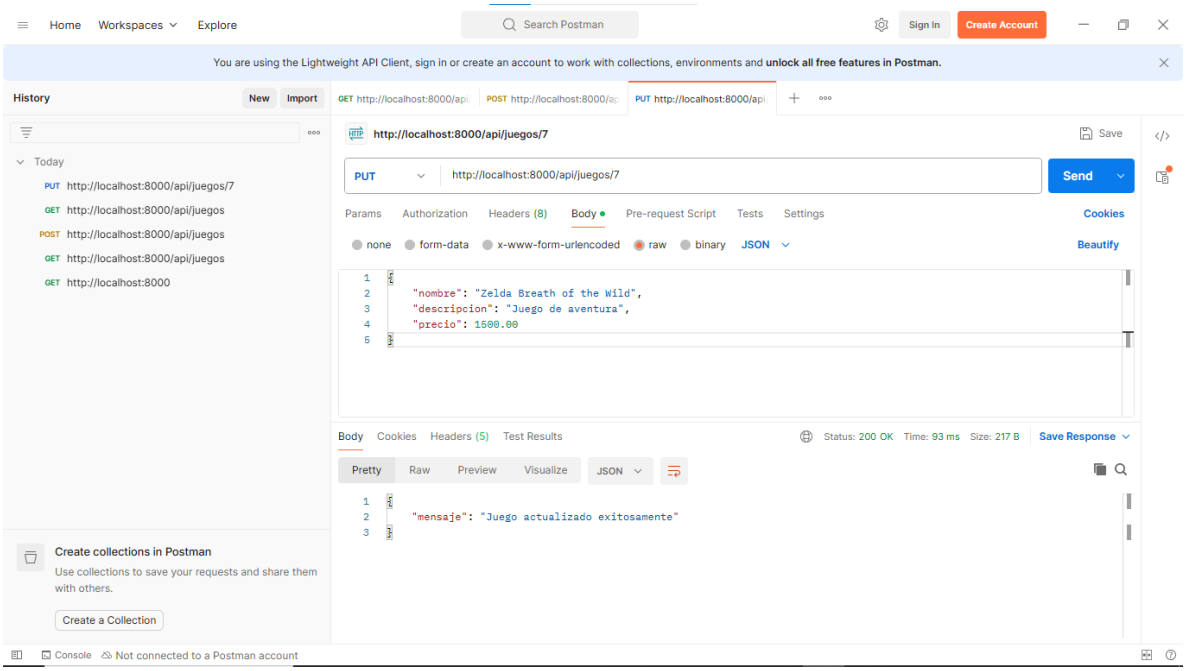
1. Obtener todos los juegos (GET /api/juegos). Este endpoint consulta la base de datos y devuelve la lista completa de videojuegos en formato JSON. En Postman, al hacer la petición, me regresó un código de estado 200 OK, lo que indica que la petición fue exitosa.



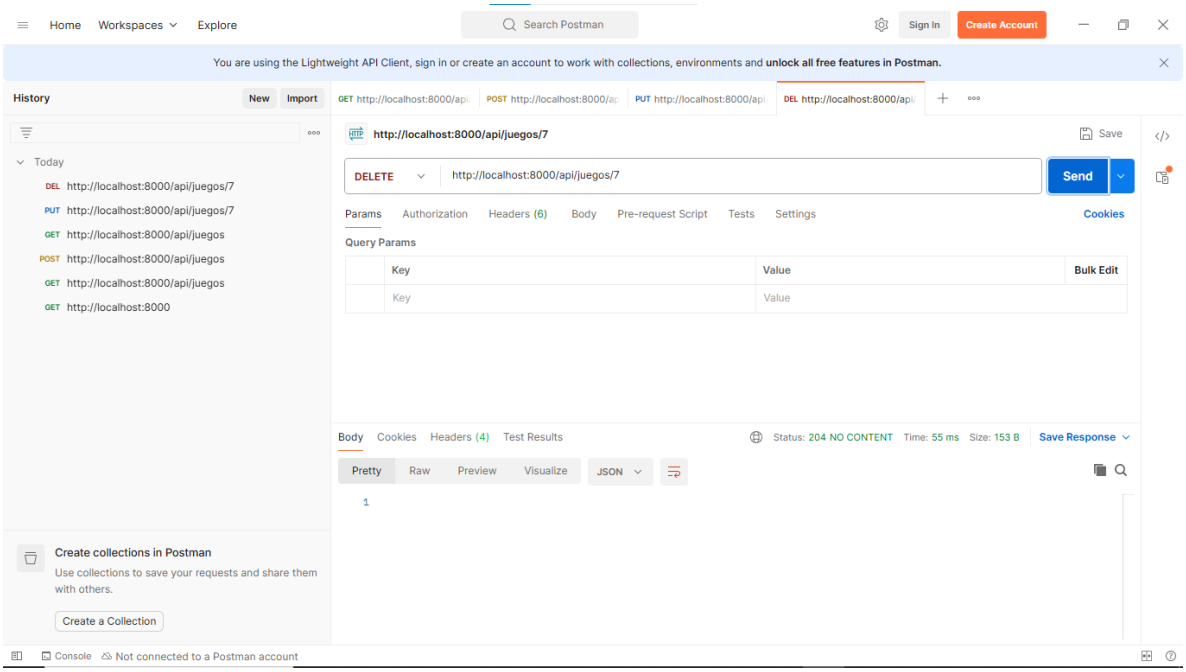
2. Crear un juego nuevo (POST /api/juegos). Este endpoint sirve para agregar un juego nuevo. Le envié los datos del juego (nombre, descripción y precio) a través del Body en formato JSON. Al enviarlo, el servidor me respondió con un código 201 Created y un mensaje de éxito.



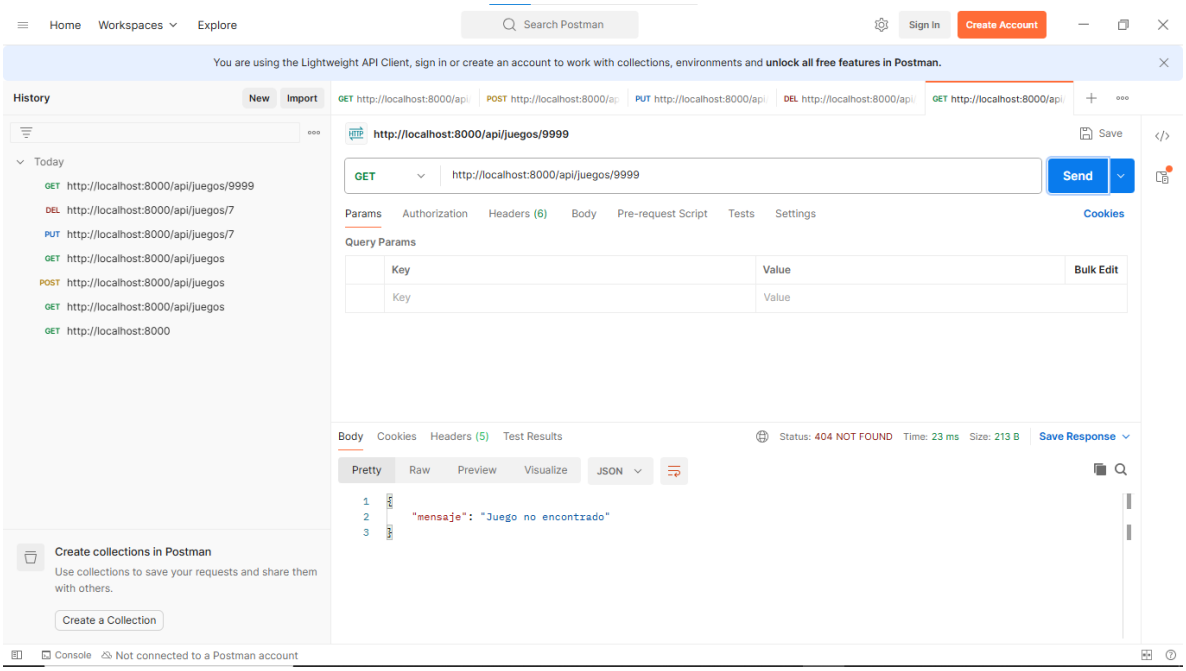
3. Modificar un juego existente (PUT /api/juegos/). Aquí probé actualizando los datos de un juego específico (el juego con el ID 7). Le mandé los datos nuevos por el Body y me devolvió un código 200 OK, confirmando que la base de datos se actualizó correctamente.



4. Eliminar un juego (DELETE /api/juegos/). Con este endpoint borré el juego que había estado modificando (ID 7). Como la acción de eliminar no necesita devolver información extra, la API me respondió correctamente con un código 204 No Content.



5. Manejo de Errores (404 Not Found). También implementé validaciones. Por ejemplo, si alguien busca o intenta interactuar con un ID de juego que no existe en la base de datos (como el ID 9999), la API devuelve un error 404 Not Found con un mensaje claro de que el juego no se encontró.



Reflexión sobre diferencias entre rutas REST manuales y Flask-RESTful.

Al realizar esta actividad, noté una gran diferencia entre hacer una API con rutas normales de Flask y usar Flask-RESTful.

Si lo hiciéramos de forma manual con @app.route(), tendríamos que usar muchos condicionales (como if request.method == 'POST':) dentro de una misma función para saber qué acción ejecutar, y además tendríamos que estar convirtiendo los datos a JSON a mano todo el tiempo.

En cambio, al usar Flask-RESTful, el código queda mucho más limpio y estructurado usando Clases (Resource). Cada método HTTP (get, post, put, delete) tiene su propia función dedicada dentro de la clase. Esto hace que el código sea más fácil de leer, de mantener y se siente mucho más profesional, ya que la librería hace gran parte del trabajo pesado por nosotros.